

CONTENTS

Programación Dinámica	
Guía de estudio con tolerancia cero — Módulo 6, Programación 3 (FING UdelaR)	
0. Por qué existe este tema	
1. El método sistemático para plantear cualquier DP	
2. Weighted Interval Scheduling (WIS)	
Ejemplo completo con 6 intervalos	
3. Knapsack (Mochila 0/1)	
Ejemplo completo ($n=4$, $W=8$)	
4. Sequence Alignment / Edit Distance	
Ejemplo completo ($X=\text{GATC}$, $Y=\text{GACC}$, $\delta=2$, $\alpha=1$)	
5. Caminos mínimos con Programación Dinámica	
6. Top-down (memoización) vs. bottom-up (tabulación)	
7. Trampas típicas de examen	
Verificación con Python	
Resumen ultra-rápido (2 minutos)	

Programación Dinámica

GUÍA DE ESTUDIO CON TOLERANCIA CERO — MÓDULO 6, PROGRAMACIÓN 3 (FING UDELAR)

0. Por qué existe este tema

Divide y Vencerás (Módulo 5) resuelve un problema partiéndolo en subproblemas **independientes**: mergesort ordena la mitad izquierda y la mitad derecha sin que una le importe nada de la otra, y por eso alcanza con resolver cada subproblema **una sola vez**. La Programación Dinámica (PD) entra en escena exactamente cuando esa independencia se rompe: los subproblemas naturales de un problema se **superponen** — el mismo subproblema chico aparece una y otra vez dentro de subproblemas más grandes distintos — y una recursión ingenua terminaría resolviéndolo miles de veces.

Subproblemas superpuestos (overlapping subproblems): distintos subproblemas “grandes” comparten subproblemas “chicos” en común. Ejemplo mínimo: Fibonacci — fib(7) llama a fib(6) y fib(5); fib(6) a su vez llama a fib(5) y fib(4). fib(5) se calcula 2 veces, fib(4) 3 veces, etc. Sin memoria, el árbol de llamadas crece exponencialmente aunque solo hay $n+1$ subproblemas distintos.

La segunda condición, igual de necesaria, es la **subestructura óptima**: la solución óptima de un problema se puede construir a partir de las soluciones óptimas de sus subproblemas. Esto es lo que permite escribir una recurrencia — sin subestructura óptima no hay ecuación que combine “lo mejor de las partes” en “lo mejor del todo”.

	Divide y Vencerás	Programación Dinámica
Subproblemas	Independientes (no se repiten)	Superpuestos (se repiten)
Cada subproblema se resuelve	1 vez	1 vez, pero se guarda el resultado
Combina resultados de	Ramas disjuntas	Subproblemas que se reutilizan entre sí
Ejemplo típico	Mergesort, mediana de dos arreglos (Práctico 5)	WIS, Knapsack, Edit Distance

Regla de oro de todo el módulo: **PD = recursión con memoria + subestructura óptima**. Si un problema tiene subestructura óptima pero sus subproblemas NO se superponen, ya es D&V (no hace falta memoria, no gana nada guardar resultados). Si se superponen pero no hay subestructura óptima (por ejemplo, “encontrar el camino simple más largo” en un grafo general), PD no aplica directamente — de hecho ese problema es NP-difícil.

1. El método sistemático para plantear cualquier DP

Este es el contenido más importante de la guía. La mayoría de los errores de examen en PD no son errores de cálculo — son errores de **planteo**: mal definido el estado, o una recurrencia que no cubre todos los casos. Seguí siempre estos seis pasos, en este orden, y escribilos explícitamente en el examen (no solo la fórmula final):

(i) Definir el estado / subproblema en palabras simples. Antes de escribir ningún símbolo, hay que poder decir en una frase qué representa $OPT(i)$ (o $OPT(i, j)$, etc.). No “ $OPT(i)$ = la recurrencia”, sino algo como: “ $OPT(i)$ = la ganancia máxima obtenible usando únicamente los primeros i intervalos”. Si no podés escribir esa frase con precisión, todavía no entendiste el problema.

(ii) Escribir la recurrencia. Relacionar el subproblema con subproblemas **estrictamente más chicos**. La técnica estándar: pensar en la solución óptima del subproblema grande y preguntarse **qué decisión se tomó en el “último paso”** (el último ítem, la última semana, el último carácter, la última arista) — y enumerar TODAS las opciones posibles en ese último paso, no solo la que “parece” mejor.

(iii) Identificar el/los caso(s) base. El o los subproblemas más chicos, donde la recurrencia ya no puede aplicarse porque no hay “subproblema más chico” al cual referirse (por ejemplo, $OPT(0) = 0$, o una cadena vacía).

(iv) Definir el orden de cómputo. Un orden topológico respecto a la recurrencia: todo subproblema debe calcularse **después** de todos los subproblemas de los que depende. En DP 1D esto casi siempre es “de menor a mayor índice”; en DP 2D, normalmente por filas o por diagonales — pero hay que verificarlo mirando de qué depende cada celda, no asumirlo.

(v) Ubicar la respuesta final. ¿En qué celda de la tabla está la respuesta al problema original? No siempre es la última celda calculada (a veces es un máximo/mínimo sobre una fila o columna entera — ver Ejercicio 3 de semana 11 en el práctico resuelto).

(vi) **Analizar la complejidad.** Complejidad total = (cantidad de subproblemas distintos) $\times$ (costo de calcular cada uno a partir de los ya calculados). Este producto es la razón profunda por la que PD puede convertir un algoritmo exponencial en uno polinomial: en lugar de un árbol de recursión con ramas repetidas exponenciales, hay una tabla de tamaño polinomial y cada celda cuesta poco.

Truco de examen: escribí siempre el paso (i) — la descripción en palabras del estado — ANTES que la fórmula. Un examen que corrige con tolerancia cero suele dar puntaje explícito a esta frase, independientemente de que la fórmula esté bien. Y si te trabás planteando la recurrencia, volver a leer en voz alta la frase del estado casi siempre destraba el paso (ii).

Error frecuente de tolerancia cero: plantear la recurrencia mirando “hacia adelante” (qué pasa después de la decisión actual) en lugar de “hacia atrás” (cuál fue la última decisión que llevó hasta acá). Casi todas las recurrencias de PD se plantean mirando hacia atrás: “¿cuál pudo haber sido el último paso para llegar a este estado, y qué subproblema queda si lo deshago?” — nunca “¿qué voy a hacer después?”.

2. Weighted Interval Scheduling (WIS)

Problema: dados n intervalos, cada uno con un tiempo de inicio s_j , tiempo de fin f_j y peso (valor) v_j , elegir un subconjunto de intervalos **que no se superpongan entre sí** (dos intervalos i, j son compatibles si $f_i \leq s_j$ o $f_j \leq s_i$) de forma que la suma de los pesos elegidos sea máxima.

Preprocesamiento obligatorio: ordenar los n intervalos por tiempo de fin creciente: $f_1 \leq f_2 \leq \dots \leq f_n$. Definir $p(j)$ = el mayor índice $i < j$ tal que el intervalo i es compatible con j (es decir, $f_i \leq s_j$); $p(j) = 0$ si no existe tal i .

(i) **Estado:** $OPT(j)$ = la ganancia máxima obtenible usando únicamente los intervalos $1..j$ (ordenados por fin).

(ii) **Recurrencia:** mirando hacia atrás, el intervalo j en la solución óptima de $OPT(j)$ o **está incluido**, o **no está incluido**:

- Si j está incluido: no se puede usar ningún intervalo incompatible con j , es decir ningún intervalo entre $p(j)+1$ y $j-1$ — la mejor ganancia posible es $v_j + OPT(p(j))$.
- Si j no está incluido: la mejor ganancia es exactamente $OPT(j-1)$.

$$\text{OPT}(j) = \max\{v_j + \text{OPT}(p(j)), \text{OPT}(j-1)\}$$

(iii) **Caso base:** $\text{OPT}(0) = 0$ (cero intervalos, cero ganancia).

(iv) **Orden de cómputo:** de $j=1$ a $j=n$ (cada $\text{OPT}(j)$ depende solo de índices menores).

(v) **Respuesta final:** $\text{OPT}(n)$.

(vi) **Complejidad:** n subproblemas, cada uno $O(1)$ una vez que $p(j)$ está precalculado (ordenar cuesta $O(n \log n)$; calcular todos los $p(j)$ con búsqueda binaria sobre los f_j ordenados cuesta $O(n \log n)$). **Total:** $O(n \log n)$.

EJEMPLO COMPLETO CON 6 INTERVALOS

Instancia (ya ordenada por f_j creciente):

j	s_j	f_j	v_j	$p(j)$
1	1	4	3	0
2	3	5	5	0
3	0	6	6	0
4	5	7	4	2
5	3	8	8	0
6	5	9	3	2

Tabla de memoización llenada a mano (columna por columna, $j=0$ a $j=6$):

j	$v_j + \text{OPT}(p(j))$	$\text{OPT}(j-1)$	$\text{OPT}(j) = \max$	¿incluye j?
0	—	—	0	—
1	$3 + \text{OPT}(0) = 3 + 0 = 3$	$\text{OPT}(0) = 0$	3	sí
2	$5 + \text{OPT}(0) = 5 + 0 = 5$	$\text{OPT}(1) = 3$	5	sí
3	$6 + \text{OPT}(0) = 6 + 0 = 6$	$\text{OPT}(2) = 5$	6	sí
4	$4 + \text{OPT}(2) = 4 + 5 = 9$	$\text{OPT}(3) = 6$	9	sí
5	$8 + \text{OPT}(0) = 8 + 0 = 8$	$\text{OPT}(4) = 9$	9	no
6	$3 + \text{OPT}(2) = 3 + 5 = 8$	$\text{OPT}(5) = 9$	9	no

Ganancia óptima: $\text{OPT}(6) = 9$.

Reconstrucción de la solución (no solo el valor): se recorre desde $j=n$ hacia atrás, comparando en cada paso si “incluir j” ($v_j + \text{OPT}(p(j))$) empató o superó a “no incluirlo” ($\text{OPT}(j-1)$):

```
FindSolution(j):
    si j == 0: devolver conjunto vacío
    si  $v[j] + \text{OPT}(p(j)) \geq \text{OPT}(j-1)$ :
        devolver FindSolution(p(j))  $\cup \{j\}$     # se incluye j
    si no:
        devolver FindSolution(j-1)              # no se incluye j
```

Aplicado a la tabla: $j=6 \rightarrow$ no incluye ($8 < 9$) $\rightarrow$ sigue en $j=5 \rightarrow$ no incluye ($8 < 9$) $\rightarrow$ sigue en $j=4 \rightarrow$ incluye ($9 \geq 6$) $\rightarrow$ sigue en $p(4)=2 \rightarrow$ incluye ($5 \geq 3$) $\rightarrow$ sigue en $p(2)=0 \rightarrow$ fin.

Solución óptima: {intervalo 2, intervalo 4}, con valor $5+4 = 9$. ✓ coincide con $\text{OPT}(6)$.

Verificación independiente con Python (fuerza bruta sobre las $2^6=64$ combinaciones de intervalos, filtrando las compatibles entre sí): confirma que el máximo valor entre subconjuntos compatibles es exactamente 9, logrado por $\{2,4\}$. Script: `verify_wis.py` (ver sección de verificación al final de esta guía).

3. Knapsack (Mochila 0/1)

Problema: n ítems, cada uno con peso w_i y valor v_i (enteros positivos), y una mochila de capacidad W . Elegir un subconjunto de ítems (cada uno se toma **entero o nada**, de ahí “0/1”) cuyo peso total no supere W y cuyo valor total sea máximo.

El error clásico al intentarlo con un estado 1D: definir $\text{OPT}(i)$ = “mejor valor usando los primeros i ítems” **no alcanza** — la decisión de incluir el ítem i depende de cuánta capacidad queda disponible, y esa capacidad depende de qué ítems previos se eligieron. Hace falta agregar la capacidad restante como **segunda dimensión** del estado.

(i) Estado: $\text{OPT}(i, w)$ = el valor máximo obtenible usando únicamente los primeros i ítems, con una capacidad de mochila disponible de exactamente w .

(ii) Recurrencia: mirando el ítem i (el “último” de los i disponibles):

- Si $w_i > w$ (no entra ni siquiera solo): forzosamente no se incluye. $\text{OPT}(i, w) = \text{OPT}(i-1, w)$.
- Si $w_i \leq w$: se elige lo mejor entre incluirlo o no:

$$\text{OPT}(i, w) = \text{OPT}(i-1, w) \text{ si } w_i > w \quad \text{OPT}(i, w) = \max\{ \text{OPT}(i-1, w), v_i + \text{OPT}(i-1, w-w_i) \} \text{ si } w_i \leq w$$

(iii) Caso base: $\text{OPT}(0, w) = 0$ para todo w (sin ítems, valor 0), y $\text{OPT}(i, 0) = 0$ para todo i (sin capacidad, valor 0).

(iv) Orden de cómputo: por filas, $i=0$ hasta n ; dentro de cada fila, $w=0$ hasta W (la fila i solo depende de la fila $i-1$ completa, ya calculada).

(v) Respuesta final: $\text{OPT}(n, W)$.

(vi) Complejidad: hay $(n+1) \times (W+1)$ celdas, cada una $O(1)$. **Total: $O(nW)$.**

⚠ Trampa de examen — complejidad pseudo-polinomial: $O(nW)$ **NO es polinomial en el tamaño de la entrada.** El tamaño de la entrada de W (la capacidad) es la cantidad de bits necesarios para representarlo, es decir $O(\log W)$, no W . Si $W = 2^{100}$, el algoritmo tarda proporcional a 2^{100} , ¡exponencial en el tamaño real de la entrada! A este tipo de algoritmo se lo llama **pseudo-polinomial**: es polinomial en el *valor numérico* de W , pero exponencial en la cantidad de *dígitos/bits* de W . Esta es una de las trampas más repetidas en los exámenes de este curso — no confundir “el algoritmo corre rápido para W chico” con “el algoritmo es polinomial”.

EJEMPLO COMPLETO (N=4, W=8)

i	peso w_i	valor v_i
1	2	3
2	3	4
3	4	5
4	5	6

Tabla OPT(i,w) llenada a mano (filas = ítems 0..4, columnas = capacidad w 0..8):

i \ w	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1 (w=2,v=3)	0	0	3	3	3	3	3	3	3
2 (w=3,v=4)	0	0	3	4	4	7	7	7	7
3 (w=4,v=5)	0	0	3	4	5	7	8	9	9
4 (w=5,v=6)	0	0	3	4	5	7	8	9	10

Ejemplo de una celda calculada a mano: OPT(4,8): $w_4=5 \leq 8$, entonces $\text{OPT}(4,8) = \max\{\text{OPT}(3,8), 6+\text{OPT}(3,3)\} = \max\{9, 6+4\} = \max\{9, 10\} = 10$.

Reconstrucción: partiendo de $(i,w)=(4,8)$, en cada paso comparar si OPT(i,w) vino de “no incluir i” ($\text{OPT}(i-1,w)$) o “incluir i” ($v_i+\text{OPT}(i-1,w-w_i)$):

- (4,8): $\text{OPT}(3,8)=9 \neq 10 = \text{OPT}(4,8) \rightarrow$ se incluyó el ítem 4. Saltar a (3, $8-5=3$).
- (3,3): $\text{OPT}(2,3)=4 = \text{OPT}(3,3) \rightarrow$ no se incluyó el ítem 3. Saltar a (2,3).
- (2,3): $\text{OPT}(1,3)=3 \neq 4 = \text{OPT}(2,3) \rightarrow$ se incluyó el ítem 2. Saltar a (1, $3-3=0$).
- (1,0): capacidad 0 $\rightarrow$ fin.

Solución óptima: {ítem 2, ítem 4}, peso $3+5=8 \leq 8 \checkmark$, valor $4+6=10 \checkmark$ (coincide con OPT(4,8)).

Verificado con Python contra fuerza bruta ($2^4=16$ subconjuntos): el máximo valor con peso ≤ 8 es 10, logrado por {ítem 2, ítem 4}. Script: `verify_knapsack.py`.

4. Sequence Alignment / Edit Distance

Problema: dadas dos cadenas X (largo m) e Y (largo n), transformar X en Y usando operaciones de **sustitución** (cambiar un carácter por otro, costo α si son distintos, 0 si coinciden), **inserción** y **borrado** (cada una con costo δ , el “costo de gap”), minimizando el costo total. Equivalentemente: encontrar el **alineamiento** de mínimo costo entre X e Y.

(i) **Estado:** $\text{OPT}(i,j)$ = costo mínimo de alinear el prefijo $X_{1..i}$ con el prefijo $Y_{1..j}$.

(ii) **Recurrencia:** mirando la **última columna** del alineamiento óptimo de $(X_{1..i}, Y_{1..j})$, hay exactamente tres posibilidades:

- Se alinean x_i con y_j (match si son iguales, mismatch con costo α si no): queda por resolver $\text{OPT}(i-1, j-1)$.
- x_i se alinea con un gap (x_i se borra): costo $\delta + \text{OPT}(i-1, j)$.
- y_j se alinea con un gap (se inserta y_j): costo $\delta + \text{OPT}(i, j-1)$.

$$\text{OPT}(i,j) = \min\{ \text{costo}(x_i, y_j) + \text{OPT}(i-1, j-1), \delta + \text{OPT}(i-1, j), \delta + \text{OPT}(i, j-1) \}$$

donde $\text{costo}(x_i, y_j) = 0$ si $x_i = y_j$, α si no.

(iii) **Caso base:** $\text{OPT}(i, 0) = i \cdot \delta$ (borrar los primeros i caracteres de X), $\text{OPT}(0, j) = j \cdot \delta$ (insertar los primeros j caracteres de Y). En particular $\text{OPT}(0, 0) = 0$.

(iv) **Orden de cómputo:** por filas o por columnas (cada celda (i, j) depende de $(i-1, j-1)$, $(i-1, j)$ y $(i, j-1)$, todas “antes” en cualquiera de los dos órdenes).

(v) **Respuesta final:** $\text{OPT}(m, n)$.

(vi) **Complejidad:** $(m+1)(n+1)$ celdas, $O(1)$ cada una. **Total:** $O(mn)$.

EJEMPLO COMPLETO (X=GATC, Y=GACC, $\Delta=2$, $A=1$)

Tabla llenada a mano (filas=X, columnas=Y; primera fila/columna = casos base):

	""	G	A	C	C
""	0	2	4	6	8
G	2	0	2	4	6
A	4	2	0	2	4
T	6	4	2	1	3
C	8	6	4	2	1

Ejemplo de celda a mano — $\text{OPT}(4,4)$ ($X_{1..4}=\text{GATC}$, $Y_{1..4}=\text{GACC}$): $x_4=\text{C}$, $y_4=\text{C} \rightarrow$ coinciden, costo=0. $\text{OPT}(4,4) = \min\{0+\text{OPT}(3,3), 2+\text{OPT}(3,4), 2+\text{OPT}(4,3)\} = \min\{0+1, 2+3, 2+2\} = \min\{1,5,4\} = 1$.

Distancia de edición óptima: $\text{OPT}(4,4) = 1$.

Reconstrucción del alineamiento (recorrer la tabla desde (m,n) hacia $(0,0)$, en cada celda identificando cuál de las tres opciones logró el mínimo):

$(4,4) \rightarrow$ diagonal (match C-C, costo 0) $\rightarrow (3,3) \rightarrow$ diagonal (T vs C, mismatch, costo 1) $\rightarrow (2,2) \rightarrow$ diagonal (match A-A) $\rightarrow (1,1) \rightarrow$ diagonal (match G-G) $\rightarrow (0,0)$.

```
X:  G A T C
Y:  G A C C
```

Costo total: 3 matches (0 cada uno) + 1 mismatch ($T \leftrightarrow C$, costo 1) = **1**, coincide con $\text{OPT}(4,4)$.

Este es el mismo esquema que usa el algoritmo **Needleman-Wunsch** en bioinformática. Si el enunciado pide explícitamente reconstruir el alineamiento (no solo la distancia), hay que guardar durante el llenado de la tabla — o recalcular al reconstruir, como se hizo acá — qué opción ganó en cada celda.

Verificado con Python: la tabla completa, la reconstrucción del alineamiento (costo recomputado = 1, coincide) y una recursión con memoización independiente (`functools.lru_cache`) que da el mismo valor. Script: `verify_edit.py`.

5. Caminos mínimos con Programación Dinámica

El material de este curso (semana 11, Ejercicio 3, basado en KT Ex. 6.22) sí incluye una variante de **caminos mínimos vía PD**, pero **no cubre Bellman-Ford completo como algoritmo aparte** — lo que se pide es una recurrencia de PD que, además del costo del camino mínimo, cuenta **cuántos** caminos de costo mínimo existen entre dos nodos de un grafo dirigido donde todo ciclo tiene costo positivo. Esta sección documenta esa variante porque es evaluable; si buscás Bellman-Ford “clásico” (solo el costo, con aristas negativas y detección de ciclos negativos), es la misma idea de PD por capas pero sin el conteo $N(i,s)$.

Idea de PD por capas: en un grafo con ciclos de costo positivo, ningún camino mínimo necesita repetir un ciclo (repetirlo solo suma costo). Por lo tanto todo camino mínimo es simple y usa **a lo sumo $n-1$ aristas** (n = cantidad de nodos). Esto permite definir el estado por “cantidad de aristas usadas”:

(i) **Estado:** $OPT(i,s)$ = costo del camino mínimo de v (nodo fijo de origen) a s usando **exactamente** i aristas. $N(i,s)$ = cantidad de caminos que logran ese costo mínimo con exactamente i aristas.

(ii) **Recurrencia:** para llegar a s en exactamente i pasos, el paso anterior tiene que haber llegado a algún nodo t (predecesor de s) en exactamente $i-1$ pasos, seguido de la arista (t,s) :

$$OPT(i,s) = \min_{(t,s) \in E} \{ OPT(i-1,t) + \text{costo}(t,s) \}$$

$$N(i,s) = \sum_{(t,s) \in E : OPT(i-1,t) + \text{costo}(t,s) = OPT(i,s)} N(i-1,t)$$

(la suma de $N(i,s)$ recorre **todos** los predecesores t que efectivamente logran el mínimo — puede haber más de uno, y cada uno aporta sus propios $N(i-1,t)$ caminos).

(iii) **Caso base:** $OPT(0,v) = 0$, $N(0,v) = 1$ (el camino vacío desde v hasta sí mismo). $OPT(0,s) = +\infty$ para $s \neq v$.

(iv) **Orden de cómputo:** $i = 1, 2, \dots, n-1$ (cada capa i depende solo de la capa $i-1$).

(v) **Respuesta final:** el costo del camino mínimo real de v a w es $OPT(w) = \min_i OPT(i,w)$ (tomando el mínimo sobre todas las cantidades de aristas posibles), y la cantidad de caminos mínimos es $N(w) = \sum_i : OPT(i,w) = OPT(w) N(i,w)$ (sumando **todas** las capas i que logran ese mínimo — puede haber caminos de distinta longitud con el mismo costo total).

(vi) **Complejidad:** $n-1$ capas $\times$ n nodos $\times$ $O(\text{grado del nodo})$ para cada transición $\rightarrow$ sumando sobre todos los nodos de una capa, cada capa cuesta $O(m)$. **Total: $O(nm)$.**

Ejemplo completo (grafo dirigido, 5 nodos 0..4, $v=0$, $w=4$): aristas $0 \rightarrow 1(2)$, $0 \rightarrow 2(5)$, $1 \rightarrow 2(1)$, $1 \rightarrow 3(2)$, $2 \rightarrow 3(1)$, $2 \rightarrow 4(7)$, $3 \rightarrow 4(2)$. Ningún ciclo (es un DAG en este ejemplo, caso particular válido).

Tabla $OPT(i,s)$, filas $i=0..4$, columnas $s=0..4$:

$i \backslash s$	0	1	2	3	4
0	0	∞	∞	∞	∞
1	∞	2	5	∞	∞
2	∞	∞	3	4	12
3	∞	∞	∞	4	6
4	∞	∞	∞	∞	6

Tabla $N(i,s)$:

$i \backslash s$	0	1	2	3	4
0	1	0	0	0	0
1	0	1	1	0	0
2	0	0	1	1	1
3	0	0	0	1	1
4	0	0	0	0	1

Cálculo de la celda $OPT(3,4)/N(3,4)$ a mano: predecesores de 4: nodo 2 (costo 7) y nodo 3 (costo 2). $OPT(3,4) = \min\{OPT(2,2)+7, OPT(2,3)+2\} = \min\{3+7, 4+2\} = \min\{10, 6\} = 6$. Solo el predecesor 3 logra el mínimo ($4+2=6$; $3+7=10$ no) $\rightarrow N(3,4) = N(2,3) = 1$.

Respuesta final: $OPT(w=4) = \min$ sobre la columna 4 = $\min\{12, 6, 6\} = 6$, logrado en las capas $i=3$ ($N=1$) e $i=4$ ($N=1$) $\rightarrow N(w) = 1+1 = 2$ caminos mínimos: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$ (costo $2+1+1+2=6$) y $0 \rightarrow 1 \rightarrow 3 \rightarrow 4$ (costo $2+2+2=6$).

Verificado con Python: DP por capas vs. enumeración por fuerza bruta de todos los caminos simples de v a w (DFS con marca de visitados) — ambos coinciden en costo mínimo (6) y cantidad de caminos mínimos (2). Script: `verify_semana11.py` (Ejercicio 3).

6. Top-down (memoización) vs. bottom-up (tabulación)

Ambos enfoques calculan **exactamente los mismos valores**, con la misma complejidad asintótica — la diferencia es de implementación, no de idea.

- **Bottom-up (tabulación):** se llena la tabla explícitamente en el orden topológico correcto (paso (iv) del método), típicamente con bucles `for` anidados. Es el enfoque usado en todos los ejemplos de esta guía.
- **Top-down (memoización):** se escribe la recursión “natural” (la que mirarías si no te importara la eficiencia), pero antes de calcular un subproblema se consulta una tabla/diccionario de caché; si ya está calculado, se devuelve directamente sin volver a recurrir.

El mismo problema (WIS) de las dos formas:

Bottom-up:

```
OPT[0] = 0
para j = 1 hasta n:
    OPT[j] = max(v[j] + OPT[p(j)], OPT[j-1])
devolver OPT[n]
```

Top-down (memoización):

```
memo = {} // vacío
función M(j):
    si j == 0: devolver 0
    si j está en memo: devolver memo[j]
    resultado = max(v[j] + M(p(j)), M(j-1))
    memo[j] = resultado
    devolver resultado
devolver M(n)
```

Ambas versiones, sobre la instancia de la sección 2, calculan exactamente los valores $OPT(0)=0$, $OPT(1)=3$, ..., $OPT(6)=9$.

	Bottom-up	Top-down
Calcula	Todos los subproblemas de la tabla, en orden	Solo los subproblemas realmente necesarios para el subproblema pedido (puede ahorrar si hay “ramas” nunca visitadas)
Riesgo típico	Ninguno particular (el orden ya está garantizado por los bucles)	Recursión profunda → riesgo de <i>stack overflow</i> en instancias grandes; hay que acordarse de revisar la memo ANTES de recurrar
Más natural para	Cuando conviene reconstruir la solución con punteros explícitos, o cuando el estado tiene un orden obvio	Cuando la recurrencia “ya está” (se dedujo mirando hacia atrás) y programarla literalmente con caché es más directo

Error frecuente de tolerancia cero: en una implementación top-down, escribir la recursión sin memoizar (sin guardar en la tabla) — eso NO es programación dinámica, es la misma recursión exponencial de siempre, aunque “parezca” PD porque se usó la recurrencia correcta. El único paso que convierte una recursión exponencial en PD es guardar (y consultar) los resultados ya calculados.

7. Trampas típicas de examen

1. No identificar bien el estado (subdimensionar). El ejemplo más claro es Knapsack: definir $OPT(i)$ sin la capacidad restante como segunda dimensión pierde información esencial — dos formas distintas de llegar al ítem i con distinta capacidad usada no son intercambiables. Regla general: el estado tiene que capturar **toda** la información necesaria para que, desde ese punto en adelante, el subproblema se pueda resolver óptimamente sin necesitar saber nada más sobre “cómo se llegó hasta ahí”.

2. Una recurrencia que no cubre todos los casos. Al mirar “hacia atrás” para el último paso, hay que enumerar TODAS las opciones posibles, no solo la más obvia. En Edit Distance son tres (match/mismatch, borrar, insertar) — olvidarse de una (por ejemplo, no considerar el gap) da una recurrencia que a veces coincide con la óptima por casualidad, pero falla en instancias donde el gap es la mejor opción.

3. Confundir la dirección de llenado de la tabla. El orden de cómputo (paso (iv)) tiene que respetar las dependencias reales de la recurrencia — hay que mirar de qué celdas depende cada casillero (no asumir “de izquierda a derecha” sin verificarlo). En DP 2D como

Knapsack o Edit Distance, llenar por columnas en vez de por filas funciona igual (ambas dependencias están disponibles antes), pero en DP con dependencias más raras (como semana 10 Ej.2, donde $OPT(i,j)$ depende de $OPT(i+1,\cdot)$) el orden es **decreciente** en i , no creciente — hay que derivarlo de la recurrencia, no memorizar “siempre de menor a mayor”.

4. Olvidar la reconstrucción de la solución cuando el ejercicio la pide. La tabla de PD por sí sola solo da el **valor** óptimo (cuánto), no **cuál** es la solución (qué elegir). Si el enunciado pide “el conjunto/plan/alineamiento óptimo” y no solo “el valor óptimo”, hay que agregar el paso de reconstrucción (guardando punteros durante el llenado, o recalculando qué opción ganó al recorrer la tabla hacia atrás, como se hizo en cada ejemplo de esta guía). Entregar solo el número cuando piden la solución es una causa frecuente de pérdida de puntos.

Verificación con Python

Todos los ejemplos de esta guía fueron verificados con scripts independientes que (a) implementan la recurrencia y confirman que reproduce exactamente los valores de la tabla a mano, y (b) comparan el óptimo contra una búsqueda por fuerza bruta sobre la instancia pequeña usada:

- `verify_wis.py` — Weighted Interval Scheduling (sección 2): DP vs. fuerza bruta sobre 2^6 subconjuntos → coinciden (óptimo = 9, solución {2,4}).
- `verify_knapsack.py` — Knapsack 0/1 (sección 3): DP vs. fuerza bruta sobre 2^4 subconjuntos → coinciden (óptimo = 10, solución {2,4}).
- `verify_edit.py` — Edit Distance (sección 4): tabla bottom-up vs. recursión con `lru_cache` (top-down) → coinciden (distancia = 1); alineamiento reconstruido recalculado da el mismo costo.
- `verify_semana11.py` (Ejercicio 3) — Caminos mínimos vía PD (sección 5): DP por capas vs. enumeración de caminos simples por DFS → coinciden (costo mínimo = 6, cantidad = 2).

No se encontraron errores en el material fuente de este módulo (Práctico semana 10 y semana 11) — los enunciados son consistentes con los algoritmos estándar del capítulo 6 de Kleinberg & Tardos.

Resumen ultra-rápido (2 minutos)

- **PD = subproblemas superpuestos + subestructura óptima.** Si los subproblemas fueran independientes, sería Divide y Vencerás; sin subestructura óptima, PD no aplica.
- **Método en 6 pasos, siempre en este orden:** (i) definir el estado en palabras, (ii) recurrencia mirando hacia atrás (último paso, TODAS las opciones), (iii) caso(s) base, (iv) orden de cómputo (topológico respecto a la recurrencia), (v) dónde está la respuesta final, (vi) complejidad = subproblemas $\times$ costo por subproblema.
- **WIS:** $OPT(j) = \max\{v_j + OPT(p(j)), OPT(j-1)\}$, $O(n \log n)$, reconstrucción comparando ambas ramas desde $j=n$ hacia atrás.
- **Knapsack 0/1:** $OPT(i, w) = \max\{OPT(i-1, w), v_i + OPT(i-1, w-w_i)\}$ si $w_i \leq w$, si no $OPT(i, w) = OPT(i-1, w)$. $O(nW)$ — **pseudo-polinomial**, no polinomial (W no es $O(\log W)$).
- **Edit Distance:** $OPT(i, j) = \min\{\text{costo}(x_i, y_j) + OPT(i-1, j-1), \delta + OPT(i-1, j), \delta + OPT(i, j-1)\}$. $O(mn)$.
- **Caminos mínimos vía PD por capas:** $OPT(i, s) = \text{costo mínimo a } s \text{ con exactamente } i \text{ aristas}$; respuesta final = mínimo sobre todas las capas i . Para contar caminos, $N(i, s)$ suma $N(i-1, t)$ sobre todos los predecesores t que logran el mínimo.
- **Top-down y bottom-up calculan lo mismo** — la diferencia es solo de implementación (recursión + caché vs. bucles explícitos). Sin memoización, una recursión “con la recurrencia correcta” sigue siendo exponencial, no es PD.
- **Las 4 trampas de examen:** estado subdimensionado, recurrencia incompleta (no cubre todos los casos del “último paso”), orden de llenado mal derivado, y olvidar la reconstrucción cuando se pide la solución (no solo el valor).